UTS

PEMBELAJARAN MESIN



NAMA : Solafide Mitchello Saba

NIM : 202531177

KELAS : H1

DOSEN : Rahma Farahningrum, S.Kom., M.Kom

NO.PC : 1

ASISTEN : 1. Muhammad Ichwan Fauzi

2. Benedict Samuel Ardy

INSTITUT TEKNOLOGI PLN

TEKNIK INFORMATIKA

2026/2027

DAFTAR ISI

DAFTAR ISI **2**
BAB I **3**
PENDAHULUAN 3
1.1 Rumusan Masalah 3
1.2 Tujuan Masalah 3
1.3 Manfaat Masalah 3
BAB II **4**
LANDASAN TEORI 4
1. Klasifikasi 4
2. Exploratory Data Analysis (EDA) 4
3. Data Pre-Processing 4
4. K-Nearest Neighbors (KNN) 5
5. Naive Bayes 5
BAB III **6**
HASIL 6
1. Persiapan dataset 7
2. EDA (Exploratory Data Analysis) 8
3. K-Nearest-Neighbors (KNN) 13
4. Naive Bayes 14
5. Perbandingan 14
BAB IV **15**
PENUTUP **15**
1. Kesimpulan Landasan Teori 15
2. Kesimpulan Praktikum 15
DAFTAR PUSTAKA **16**

BAB I

PENDAHULUAN

1.1 Rumusan Masalah

1. Bagaimana cara menerapkan algoritma K-Nearest Neighbors (KNN) dan Naive Bayes untuk melakukan prediksi diabetes?
2. Algoritma manakah yang memiliki akurasi lebih baik dalam melakukan prediksi diabetes berdasarkan hasil evaluasi model?

1.2 Tujuan Masalah

1. Memahami dan menerapkan algoritma K-Nearest Neighbors (KNN) dan Naive Bayes untuk melakukan prediksi diabetes.
2. Mengetahui dan membandingkan akurasi algoritma KNN dan Naive Bayes berdasarkan hasil evaluasi model.

1.3 Manfaat Masalah

1. Menambah pemahaman mengenai penerapan algoritma Machine Learning, khususnya KNN dan Naive Bayes, dalam melakukan klasifikasi data kesehatan.
2. Memahami proses pengolahan data, preprocessing, pelatihan model, serta pengujian hasil prediksi menggunakan Machine Learning.

BAB II

LANDASAN TEORI

1. Klasifikasi

Klasifikasi merupakan salah satu metode dalam Machine Learning yang digunakan untuk mengelompokkan data ke dalam kategori atau kelas tertentu berdasarkan karakteristik yang dimiliki oleh data tersebut. Klasifikasi termasuk ke dalam metode Supervised Learning, yaitu proses pembelajaran mesin yang menggunakan data yang telah memiliki label atau kategori sebagai acuan dalam proses pelatihan. Melalui proses tersebut, model dapat mempelajari pola dan hubungan antara karakteristik data dengan kelas yang telah ditentukan, sehingga dapat digunakan untuk mengklasifikasikan data baru. (Kittani & Abdulazeez, 2024) .

Dalam proses klasifikasi, terdapat dua komponen utama, yaitu feature dan class label. Feature merupakan atribut atau karakteristik yang digunakan sebagai dasar untuk melakukan klasifikasi, sedangkan class label merupakan kategori atau hasil klasifikasi yang ingin diprediksi. Model akan mempelajari hubungan antara feature dan class label melalui data pelatihan, kemudian menggunakan pola yang telah dipelajari untuk menentukan kelas dari data yang belum diketahui kategorinya (Haviluddin et al., 2022).

2. Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) merupakan proses awal dalam analisis data yang bertujuan untuk memahami karakteristik, struktur, dan pola yang terdapat dalam suatu dataset sebelum dilakukan proses pemodelan. EDA dilakukan dengan mengeksplorasi data untuk menemukan informasi awal serta mengetahui kondisi dan karakteristik data yang akan digunakan dalam proses analisis (Rahman & Munir, 2024).

Dalam proses EDA, terdapat beberapa teknik yang umum digunakan, seperti statistik deskriptif dan visualisasi data. Statistik deskriptif digunakan untuk mengetahui informasi seperti nilai rata-rata, nilai minimum, nilai maksimum, dan persebaran data. Sementara itu, visualisasi data menggunakan grafik seperti histogram dan boxplot untuk membantu melihat distribusi data, pola tertentu, serta kemungkinan adanya outlier atau nilai yang berbeda jauh dari sebagian besar data. Melalui teknik-teknik tersebut, kondisi dataset dapat dipahami dengan lebih baik sebelum memasuki tahap pengolahan dan pemodelan (Han & Kwak, 2023)

3. Data Pre-Processing

Data preprocessing merupakan tahapan dalam pengolahan data yang bertujuan untuk mempersiapkan dataset agar dapat digunakan dalam proses pemodelan Machine Learning. Tahapan ini dilakukan untuk menangani berbagai permasalahan pada data, seperti nilai yang hilang, data duplikat, serta perbedaan format dan tipe data. Proses preprocessing penting

karena kualitas data yang digunakan dapat memengaruhi hasil dan akurasi model yang dibangun (Koukaras & Tjortjis, 2025).

Beberapa teknik yang umum digunakan dalam data preprocessing adalah data cleaning, encoding, feature scaling, dan data splitting. Data cleaning digunakan untuk memperbaiki atau menghapus data yang tidak sesuai, sedangkan encoding mengubah data kategorikal menjadi bentuk numerik agar dapat diproses oleh algoritma. Feature scaling digunakan untuk menyamakan skala nilai antar fitur, sementara data splitting membagi dataset menjadi data pelatihan (training data) dan data pengujian (testing data). Pembagian tersebut bertujuan agar model dapat dilatih menggunakan satu bagian data dan diuji menggunakan data yang berbeda (Koukaras & Tjortjis, 2025).

4. K-Nearest Neighbors (KNN)

K-Nearest Neighbors (KNN) merupakan salah satu algoritma Supervised Learning yang digunakan untuk melakukan klasifikasi berdasarkan kedekatan karakteristik antar data. Algoritma ini bekerja dengan membandingkan data baru terhadap data yang sudah memiliki label, kemudian mencari sejumlah tetangga terdekat berdasarkan perhitungan jarak. Salah satu metode perhitungan jarak yang umum digunakan adalah Euclidean Distance, yaitu perhitungan jarak berdasarkan selisih nilai dari setiap fitur yang dimiliki oleh dua data (Halder et al., 2024).

Dalam proses klasifikasi, KNN menentukan kelas dari data baru berdasarkan mayoritas kelas yang dimiliki oleh sejumlah tetangga terdekatnya. Jumlah tetangga yang digunakan ditentukan melalui parameter K. Sebagai contoh, apabila nilai K adalah 5, algoritma akan mengambil lima data terdekat dan menentukan hasil klasifikasi berdasarkan kelas yang paling banyak muncul di antara kelima data tersebut. Pemilihan nilai K dapat mempengaruhi hasil prediksi, karena jumlah tetangga yang berbeda dapat menghasilkan keputusan klasifikasi yang berbeda pula (Halder et al., 2024).

5. Naive Bayes

Naive Bayes merupakan algoritma Supervised Learning yang digunakan untuk melakukan klasifikasi menggunakan probabilitas. Algoritma ini menggunakan Teorema Bayes untuk menghitung kemungkinan suatu data termasuk ke dalam kelas tertentu berdasarkan karakteristik atau fitur yang dimilikinya. Hasil klasifikasi ditentukan dengan memilih kelas yang memiliki nilai probabilitas paling tinggi berdasarkan perhitungan tersebut (Haviluddin et al., 2022).

Naive Bayes memiliki asumsi bahwa setiap fitur bersifat saling independen atau tidak bergantung satu sama lain ketika berada dalam suatu kelas. Asumsi ini digunakan untuk menyederhanakan perhitungan probabilitas sehingga proses klasifikasi dapat dilakukan dengan lebih sederhana dan efisien. Meskipun dalam data sebenarnya hubungan antar fitur mungkin tetap ada, algoritma ini masih dapat memberikan hasil klasifikasi yang baik pada berbagai permasalahan (Haviluddin et al., 2022).

BAB III

HASIL

```
[22]: import pandas as pd
      import matplotlib.pyplot as plt
      import seaborn as sns

      from sklearn.model_selection import train_test_split
      from sklearn.neighbors import KNeighborsClassifier
      from sklearn.naive_bayes import GaussianNB
      from sklearn.compose import ColumnTransformer
      from sklearn.preprocessing import OneHotEncoder, StandardScaler
      from sklearn.metrics import (accuracy_score)
      #202531177
```

Sebelum memulai pembuatan algoritma KNN dan Naive Bayes library-library yang dibutuhkan seperti Pandas, Seaborn, dan lainnya diimport ke dalam program untuk mempermudah pembuatan algoritma kedepannya. Fungsi dari masing-masing library adalah sebagai berikut:

Nama	**Fungsi**
Pandas	Pembacaan dan manajemen data
Matplotlib	Membuat visualisasi
Seaborn	Membuat grafik statistika
Train_test_split	Membagi data menjadi data train dan data test
KNeighborsClassifier	Menerapkan KNN
GaussianNB	Menerapkan Naive Bayes
ColumnTransformer	Menerapkan metode-metode pre-processing pada kolom data
OneHotEncoder	Mengubah data menjadi data numerik biner (0/1)
StandardScaler	Menyamakan skala nilai pada fitur numerik
Accuracy_score	Menghitung akurasi model

1. Persiapan dataset

```
[40]: df = pd.read_csv("02_diabetes.csv")

print(df.head())
print(df.shape)

#202531177
```

```
   gender   age  hypertension  heart_disease smoking_history    bmi  \
0  Female  80.0             0              1           never  25.19
1  Female  54.0             0              0         No Info  27.32
2    Male  28.0             0              0           never  27.32
3  Female  36.0             0              0         current  23.45
4    Male  76.0             1              1         current  20.14

   HbA1c_level  blood_glucose_level  diabetes
0          6.6                  140         0
1          6.6                   80         0
2          5.7                  158         0
3          5.0                  155         0
4          4.8                  155         0
(100000, 9)
```

Cell ini berfungsi untuk memeriksa apakah dataset sudah diimpor dan bisa diakses dengan benar. df.head berfungsi untuk menunjukkan 5 baris awal pada dataset dan df.shape berfungsi untuk menunjukkan dimensi dari dataset, dimana dimensi yang ditemukan adalah 100000 baris dan 9 kolom data, Dataset yang digunakan adalah data nomor 2 yaitu diabetes sesuai dengan instruksi dan pembagian dataset yang diberikan.

```
[42]: df.info()
display(df.describe())
print("\nNilai Nuull")
df.isnull().sum()
#202531177
```

```
<class 'pandas.DataFrame'>
RangeIndex: 100000 entries, 0 to 99999
Data columns (total 9 columns):
 #   Column               Non-Null Count   Dtype
---  ------               --------------   -----
 0   gender               100000 non-null  str
 1   age                  100000 non-null  float64
 2   hypertension         100000 non-null  int64
 3   heart_disease        100000 non-null  int64
 4   smoking_history      100000 non-null  str
 5   bmi                  100000 non-null  float64
 6   HbA1c_level          100000 non-null  float64
 7   blood_glucose_level  100000 non-null  int64
 8   diabetes             100000 non-null  int64
dtypes: float64(3), int64(4), str(2)
memory usage: 8.0 MB
```

	age	hypertension	heart_disease	bmi	HbA1c_level	blood_glucose_level	diabetes
count	100000.000000	100000.00000	100000.000000	100000.000000	100000.000000	100000.000000	100000.000000
mean	41.885856	0.07485	0.039420	27.320767	5.527507	138.058060	0.085000
std	22.516840	0.26315	0.194593	6.636783	1.070672	40.708136	0.278883
min	0.080000	0.00000	0.000000	10.010000	3.500000	80.000000	0.000000
25%	24.000000	0.00000	0.000000	23.630000	4.800000	100.000000	0.000000
50%	43.000000	0.00000	0.000000	27.320000	5.800000	140.000000	0.000000
75%	60.000000	0.00000	0.000000	29.580000	6.200000	159.000000	0.000000
max	80.000000	1.00000	1.000000	95.690000	9.000000	300.000000	1.000000

```
Nilai Nuull
[42]: gender               0
age                  0
hypertension         0
heart_disease        0
smoking_history      0
bmi                  0
HbA1c_level          0
blood_glucose_level  0
diabetes             0
dtype: int64
```

Cell ini berfungsi untuk melihat dengan lebih detail properti dari data-data didalam dataset seperti nilai yang kosong atau rata-rata setiap kolom dan lain-lain. df.info menunjukkan jumlah data didalam sebuah kolom seperti kolom “age” atau kolom “gender” serta memberitahu tipe data dari data-data pada kolom tersebut. df.describe berfungsi memberikan analisis statistika seperti jumlah data pada setiap kolom, rata-rata semua data pada setiap kolom, nilai tertinggi atau terendah, dan lain-lain. df.isnull().sum berfungsi menunjukkan jika ada data yang bernilai null atau kosong pada salah satu kolom. Pada dataset ini dapat dilihat bahwa tidak ada data yang bernilai null.

2. EDA (Exploratory Data Analysis)

```
[4]: print("Gender Categories")
print(df['gender'].value_counts())

print("\nSmoking History Categories")
print(df['smoking_history'].value_counts())

print("\nHypertension Distribution")
print(df['hypertension'].value_counts())

print("\nHeart Disease Distribution")
print(df['heart_disease'].value_counts())

print("\nDiabetes Distribution (Target)")
print(df['diabetes'].value_counts())

# 202531177
```

```
Gender Categories
gender
Female    58552
Male      41430
Other        18
Name: count, dtype: int64

Smoking History Categories
smoking_history
No Info        35816
never          35095
former          9352
current         9286
not current     6447
ever            4004
Name: count, dtype: int64

Hypertension Distribution
hypertension
0    92515
1     7485
Name: count, dtype: int64

Heart Disease Distribution
heart_disease
0    96058
1     3942
Name: count, dtype: int64

Diabetes Distribution (Target)
diabetes
0    91500
1     8500
Name: count, dtype: int64
```

Cell ini berfungsi untuk melihat distribusi atau jumlah data pada setiap kategori menggunakan fungsi value_counts(). Pada kolom gender, kategori Female memiliki jumlah data terbanyak yaitu 58.552, diikuti Male sebanyak 41.430 dan Other sebanyak 18. Pada kolom smoking_history, kategori No Info memiliki jumlah tertinggi yaitu 35.816, sedangkan kategori lainnya terdiri dari never, former, current, not current, dan ever. Selanjutnya, pada kolom hypertension dan heart_disease dapat dilihat bahwa sebagian besar data memiliki nilai 0, yang menunjukkan bahwa mayoritas data tidak memiliki kedua kondisi tersebut. Pada kolom diabetes yang merupakan target prediksi, terdapat 91.500 data dengan nilai 0 dan 8.500 data dengan nilai 1. Hasil ini menunjukkan bahwa jumlah data dengan kategori tidak diabetes lebih banyak dibandingkan data dengan kategori diabetes.

[26]:
```
plt.figure(figsize=(12, 8))

plt.subplot(2, 2, 1)
sns.histplot(df['age'], bins=30, kde=True)
plt.title('Age Distribution')

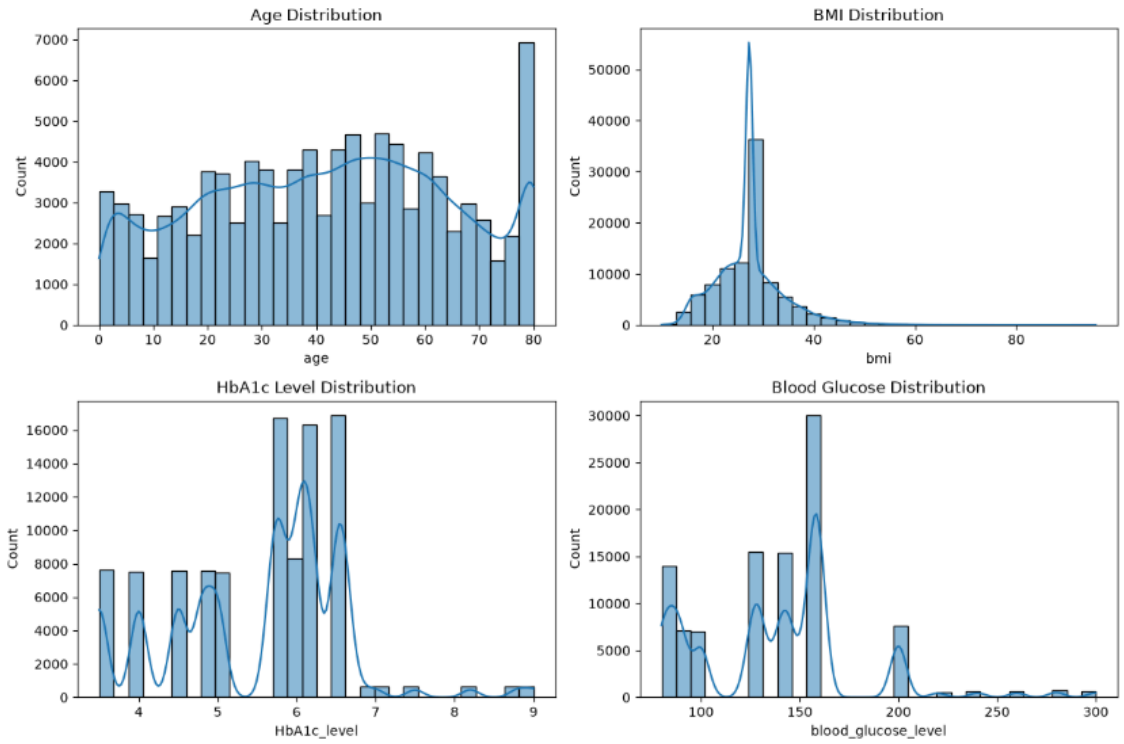
plt.subplot(2, 2, 2)
sns.histplot(df['bmi'], bins=30, kde=True)
plt.title('BMI Distribution')

plt.subplot(2, 2, 3)
sns.histplot(df['HbA1c_level'], bins=30, kde=True)
plt.title('HbA1c Level Distribution')

plt.subplot(2, 2, 4)
sns.histplot(df['blood_glucose_level'], bins=30, kde=True)
plt.title('Blood Glucose Distribution')

plt.tight_layout()
plt.show()

# 202531177
```



Cell selanjutnya pada bagian EDA berfungsi untuk membuat empat histogram berdasarkan kolom numerik yang terdapat pada dataset, yaitu Age, BMI, HbA1c, dan Blood Glucose. Histogram digunakan untuk melihat persebaran data pada setiap kolom. Dari histogram Age dapat dilihat bahwa data tersebar pada berbagai rentang usia, tetapi terdapat peningkatan jumlah data pada usia sekitar 80 tahun. Pada histogram BMI, sebagian besar data terkonsentrasi di sekitar nilai 25–30. Histogram HbA1c menunjukkan beberapa kelompok nilai dengan jumlah data yang cukup tinggi, terutama pada kisaran 6–7. Sementara itu, histogram Blood Glucose menunjukkan bahwa sebagian besar data berada pada rentang sekitar 100–160, dengan jumlah tertinggi berada di sekitar 155–160.

[6]:
```
plt.figure(figsize=(12, 8))

plt.subplot(2, 2, 1)
sns.boxplot(y=df['age'])
plt.title('Age Boxplot')

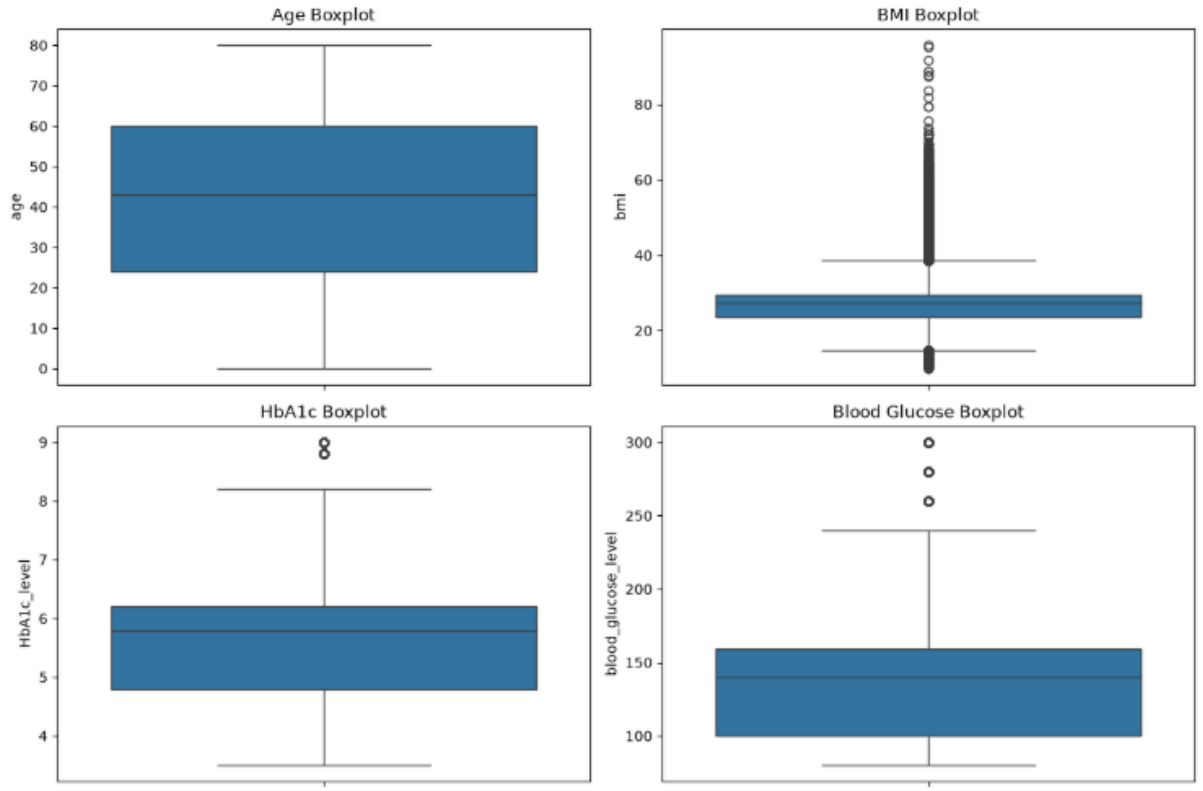
plt.subplot(2, 2, 2)
sns.boxplot(y=df['bmi'])
plt.title('BMI Boxplot')

plt.subplot(2, 2, 3)
sns.boxplot(y=df['HbA1c_level'])
plt.title('HbA1c Boxplot')

plt.subplot(2, 2, 4)
sns.boxplot(y=df['blood_glucose_level'])
plt.title('Blood Glucose Boxplot')

plt.tight_layout()
plt.show()

# 202531177
```



Cell ini berfungsi untuk menampilkan boxplot dari empat kolom numerik, yaitu age, BMI, HbA1c level, dan blood glucose level. Boxplot digunakan untuk melihat persebaran data melalui nilai median, kuartil, serta memberitahu jika terdapat outlier. Titik yang berada di luar batas whisker menunjukkan nilai yang berpotensi menjadi outlier. Dari visualisasi tersebut, terlihat bahwa beberapa kolom memiliki nilai yang berada cukup jauh dari sebagian besar data, terutama pada kolom BMI, HbA1c level, dan blood glucose level.

```
[19]: columns = ['age', 'bmi', 'HbA1c_level', 'blood_glucose_level']

for col in columns:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)

    IQR = Q3 - Q1

    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR

    outliers = df[(df[col] < lower_bound) | (df[col] > upper_bound)]

    print(f"\n{col}")
    print("Batas Bawah:", lower_bound)
    print("Batas Atas:", upper_bound)
    print("Jumlah Outlier:", len(outliers))

# 202531177
```

```
age
Batas Bawah: -28.5
Batas Atas: 111.5
Jumlah Outlier: 0

bmi
Batas Bawah: 13.709999999999997
Batas Atas: 39.55
Jumlah Outlier: 5354

HbA1c_level
Batas Bawah: 2.6999999999999993
Batas Atas: 8.3
Jumlah Outlier: 1312

blood_glucose_level
Batas Bawah: 11.5
Batas Atas: 247.5
Jumlah Outlier: 2031
```

Cell ini berfungsi untuk mencari jumlah outlier yang terlihat pada visualisasi boxplot sebelumnya. Menggunakan sebuah for loop, program akan memeriksa satu persatu kolom dan melihat berapa nilai yang berada diluar batas bawah dan batas atas dari kolom tersebut. Hasil yang ditemukan adalah kolom age tidak memiliki outlier, kolom bmi memiliki 5354 outlier, kolom HbA1c level memiliki 1312 outlier, dan kolom blood glucose level memiliki 2031 outlier. Data outlier ini tidak diubah atau dihapus karena data-data ini bisa menjadi hal yang valid dalam dunia kesehatan dan perubahan pada data dapat membuat hasil kurang merepresentasikan kejadian sebenarnya.

```
[8]: print("Duplicate Records:")
     print(df.duplicated().sum())

     # 202531177
```

```
Duplicate Records:
3854
```

```
3854
```

```
[9]: print("Dataset Awal:", df.shape)

     df = df.drop_duplicates()

     print("Dataset setelah data duplikat dihapus:", df.shape)

     # 202531177
```

```
Dataset Awal: (100000, 9)
Dataset setelah data duplikat dihapus: (96146, 9)
```

Dilakukan juga pengecekan data duplikat pada cell selanjutnya yang menunjukkan bahwa terdapat 3854 data yang duplikat yang kemudian dihapus pada sel selanjutnya.

```
[10]: X = df.drop('diabetes', axis=1)
      y = df['diabetes']

      print("Features (X):")
      display(X.head())

      print("\nTarget (y):")
      display(y.head())

      print("\nFeature:", X.shape)
      print("Target:", y.shape)

      # 202531177
```

```
Features (X):
```

	gender	age	hypertension	heart_disease	smoking_history	bmi	HbA1c_level	blood_glucose_level
0	Female	80.0	0	1	never	25.19	6.6	140
1	Female	54.0	0	0	No Info	27.32	6.6	80
2	Male	28.0	0	0	never	27.32	5.7	158
3	Female	36.0	0	0	current	23.45	5.0	155
4	Male	76.0	1	1	current	20.14	4.8	155

```
Target (y):
0    0
1    0
2    0
3    0
4    0
Name: diabetes, dtype: int64

Feature: (96146, 8)
Target: (96146,)
```

Pada cell ini dilakukan pemisahan variabel bebas dan variabel dependen dan variabel bebas (x) adalah semua kolom kecuali kolom diabetes dan variabel dependen adalah kolom diabetes. Hal ini dilakukan karena variabel diabetes adalah target atau hal yang ingin diprediksi oleh model.

```
[11]: X_train, X_test, y_train, y_test = train_test_split(
          X,
          y,
          test_size=0.2,
          random_state=42,
          stratify=y
      )

      print("Training Features:", X_train.shape)
      print("Testing Features:", X_test.shape)

      print("\nTraining Target:", y_train.shape)
      print("Testing Target:", y_test.shape)

      # 202531177
```

```
Training Features: (76916, 8)
Testing Features: (19230, 8)

Training Target: (76916,)
Testing Target: (19230,)
```

Pada cell ini dilakukan pembagian dataset menjadi data training dan data testing menggunakan fungsi train_test_split. Data training digunakan untuk melatih model, sedangkan data testing digunakan untuk menguji hasil prediksi model menggunakan data yang belum digunakan selama proses pelatihan. Parameter test_size menunjukkan bahwa 20% data digunakan sebagai data testing dan 80% sisanya digunakan sebagai data training. Parameter random_state=42 digunakan agar pembagian data dapat menghasilkan susunan yang konsisten ketika program dijalankan kembali, sedangkan stratify=y membantu mempertahankan perbandingan jumlah kelas pada kedua bagian data

```
[12]: categorical_features = ['gender', 'smoking_history']

numerical_features = [
    'age',
    'hypertension',
    'heart_disease',
    'bmi',
    'HbA1c_level',
    'blood_glucose_level'
]

preprocessor = ColumnTransformer(
    transformers=[
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features),
        ('num', StandardScaler(), numerical_features)
    ]
)

# 202531177
```

Pada cell ini ditentukan metode preprocessing yang akan diterapkan pada setiap kolom dataset. Kolom gender dan smoking_history termasuk data kategorikal sehingga akan diubah menjadi bentuk numerik menggunakan OneHotEncoder. Sementara itu, kolom numerik seperti age, hypertension, heart_disease, BMI, HbA1c level, dan blood glucose level akan diproses menggunakan StandardScaler untuk menyamakan skala nilainya. Proses ini diperlukan agar setiap fitur dapat digunakan oleh program dalam bentuk yang sesuai.

```
[13]: X_train_processed = preprocessor.fit_transform(X_train)

X_test_processed = preprocessor.transform(X_test)

print("Training Data Shape:", X_train_processed.shape)
print("Testing Data Shape:", X_test_processed.shape)

# 202531177
```

```
Training Data Shape: (76916, 15)
Testing Data Shape: (19230, 15)
```

Cell ini berfungsi untuk menerapkan preprocessing yang telah ditentukan sebelumnya terhadap data training dan data testing. Fungsi fit_transform digunakan pada data training untuk mempelajari parameter preprocessing sekaligus mengubah data. Selanjutnya, fungsi transform digunakan pada data testing dengan menggunakan parameter yang telah diperoleh dari data training. Dengan cara ini, proses pengolahan data testing tetap konsisten tanpa mempelajari parameter baru dari data tersebut. Setelah preprocessing, jumlah fitur berubah dari 8 menjadi 15 kolom karena proses OneHotEncoder mengubah data kategorikal menjadi beberapa kolom numerik.

3. K-Nearest-Neighbors (KNN)

```
[14]: knn = KNeighborsClassifier(n_neighbors=5)

knn.fit(X_train_processed, y_train)

y_pred_knn = knn.predict(X_test_processed)

# 202531177
```

Pada cell ini dilakukan pembuatan dan pelatihan model menggunakan algoritma K-Nearest Neighbors. n_neighbors=5 menunjukkan bahwa model menggunakan lima tetangga terdekat untuk menentukan kelas dari suatu data. Fungsi fit digunakan untuk melatih model menggunakan data training yang telah melalui preprocessing. Setelah proses pelatihan selesai, fungsi predict digunakan untuk menghasilkan prediksi kelas diabetes pada data testing.

```
[15]: print("Akurasi KNN:")
      print(accuracy_score(y_test, y_pred_knn))

      # 202531177

Akurasi KNN:
0.9592823712948518
```

Cell ini berfungsi untuk menghitung tingkat akurasi model KNN menggunakan fungsi accuracy_score. Berdasarkan hasil pengujian, model KNN memperoleh akurasi sebesar 0,9593 atau 95,93%. Hal ini menunjukkan bahwa sekitar 95,93% prediksi yang dihasilkan oleh model sesuai dengan label sebenarnya pada data testing.

4. Naive Bayes

```
[16]: nb = GaussianNB()

      nb.fit(X_train_processed, y_train)

      y_pred_nb = nb.predict(X_test_processed)

      # 202531177
```

Pada cell ini dibuat dan dilatih model menggunakan algoritma Gaussian Naive Bayes melalui fungsi GaussianNB(). Model kemudian dilatih menggunakan data training yang telah melalui proses preprocessing dengan fungsi fit. Setelah proses pelatihan selesai, fungsi predict digunakan untuk menghasilkan prediksi kelas diabetes pada data testing.

```
[17]: print("Akurasi Naive Bayes:")
      print(accuracy_score(y_test, y_pred_nb))

      # 202531177

Akurasi Naive Bayes:
0.6227769110764431
```

Cell ini digunakan untuk menghitung tingkat akurasi model Naive Bayes menggunakan fungsi accuracy_score. Berdasarkan hasil pengujian, model memperoleh akurasi sebesar 0,6228 atau 62,28%. Hasil tersebut menunjukkan bahwa 62,28% data prediksi yang dihasilkan oleh model sesuai dengan label sebenarnya pada data testing.

5. Perbandingan

```
[18]: comparison = pd.DataFrame({
          'KNN': {
              'Akurasi': accuracy_score(y_test, y_pred_knn),

          },
          'Naive Bayes': {
              'Akurasi': accuracy_score(y_test, y_pred_nb),

          }
      })

      display(comparison.round(4))

      # 202531177
```

	KNN	Naive Bayes
Akurasi	0.9593	0.6228

Cell ini digunakan untuk menampilkan perbandingan tingkat akurasi dari kedua algoritma yang telah diterapkan. Berdasarkan hasil yang diperoleh, KNN menghasilkan akurasi sebesar 95,93%, sedangkan Naive Bayes menghasilkan akurasi sebesar 62,28%. Dengan demikian, dapat dilihat bahwa KNN memiliki persentase yang lebih tinggi dibandingkan Naive Bayes pada pengujian dataset ini.

BAB IV
PENUTUP

1. Kesimpulan Landasan Teori

Berdasarkan landasan teori yang telah dibahas, klasifikasi merupakan salah satu metode Supervised Learning yang digunakan untuk mengelompokkan data berdasarkan karakteristik tertentu. Sebelum proses klasifikasi dilakukan, Exploratory Data Analysis (EDA) dapat digunakan untuk memahami karakteristik dan pola dalam dataset, sedangkan data preprocessing bertujuan untuk mempersiapkan data agar dapat digunakan dalam pemodelan. Algoritma K-Nearest Neighbors (KNN) melakukan klasifikasi berdasarkan kedekatan antar data, sementara Naive Bayes menggunakan perhitungan probabilitas berdasarkan Teorema Bayes.

2. Kesimpulan Praktikum

Berdasarkan hasil pengujian, algoritma KNN memperoleh tingkat akurasi sebesar 95,93%, sedangkan algoritma Naive Bayes memperoleh akurasi sebesar 62,28%. Dari hasil tersebut, KNN menghasilkan akurasi yang lebih tinggi dibandingkan Naive Bayes pada dataset dan pembagian data yang digunakan dalam praktikum ini. Praktikum ini memberikan pemahaman mengenai proses pengolahan dataset, penerapan algoritma klasifikasi, serta perbandingan hasil prediksi menggunakan tingkat akurasi.

DAFTAR PUSTAKA

Halder, R. K., Uddin, M. N., Uddin, M. A., Aryal, S., & Khraisat, A. (2024). Enhancing K-nearest neighbor algorithm: A comprehensive review and performance analysis of modifications. *Journal of Big Data, 11*, Article 113. https://link.springer.com/article/10.1186/s40537-024-00973-y

Haviluddin, Puspitasari, N., Burhandeny, A. E., Nurulita, A. D. A., & Trahutomo, D. (2022). Naïve Bayes and K-Nearest Neighbor algorithms performance comparison in diabetes mellitus early diagnosis. *International Journal of Online and Biomedical Engineering, 18*(15), 202–215. https://online-journals.org/index.php/i-joe/article/view/34143

Han, S., & Kwak, I.-Y. (2023). Mastering data visualization with Python: Practical tips for researchers. *Journal of Minimally Invasive Surgery, 26*(4), 167–175. https://doi.org/10.7602/jmis.2023.26.4.167

Koukaras, P., & Tjortjis, C. (2025). Data preprocessing and feature engineering for data mining: Techniques, tools, and best practices. *AI, 6*(10), Article 257. https://www.mdpi.com/2673-2688/6/10/257

Kittani, T., & Abdulazeez, A. (2024). Deep learning classification algorithms applications: A review. *Indonesian Journal of Computer Science, 13*(3), 4287–4311. https://www.ijcs.net/ijcs/index.php/ijcs/article/view/4064

Rahman, D. F., & Munir, A. Q. (2024). Pendekatan eksploratif dalam analisis data harga pangan nasional dengan Tableau. *ILKOMNIKA: Journal of Computer Science and Applied Informatics, 6*(2), 168–178. https://journal.unublitar.ac.id/ilkomnika/index.php/ilkomnika/article/view/659